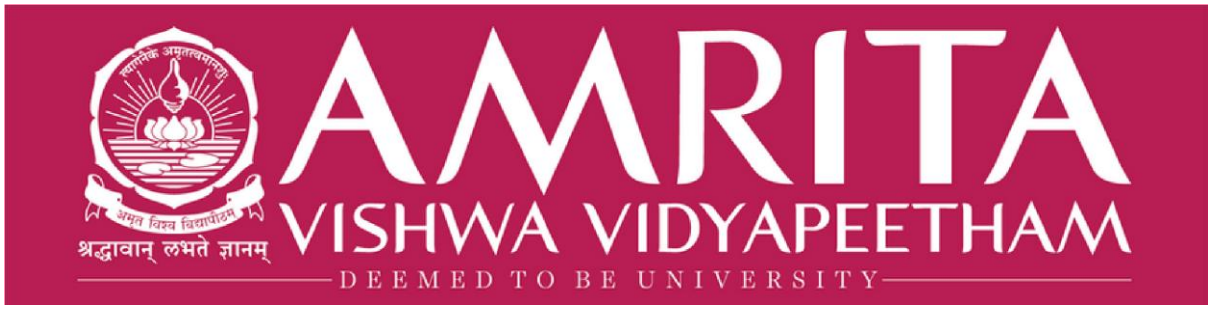


DATA STRUCTURES AND ALGORITHMS
23CSE203 LAB MANUAL

Submitted By

NAME	T. Ramcharan Reddy
ROLL NO	AV.SC.U4CSE25149
YEAR/SEM/SECTION	1ST YEAR/ SEM-3/CSE-B
CREDITS	04

Submitted to

NAME	DR. RAJ KUMAR BATCHU
DESIGNATION	ASST. PROFESSOR (SI. GRADE)
DEPARTMENT	SCHOOL OF COMPUTING

Index

S.NO	Title	Page no	Signature
1	Week1 (a). A rocket launch system displays a countdown before launch. Instead of loop implement countdown using recursion. Write a recursive Python program to display a countdown from n to 1, followed by "Launch!". (b) Banks calculate the growth of an investment using compound interest. The formulae require calculating powers, which can be implemented recursively. Write a recursive program to calculate P^N where: P=Principal Growth, N=number of years. (c) An HR department stores ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in the list. (d) In logistics company, develop a recursive program to calculate the factorial of a given number. (e) Write a recursive program to print the first n terms of the Fibonacci series	6-10	
2	1. Write the implementation for the linear search algorithm	11-13	

	2. Write the implementation for the binary search algorithm 3. Write the implementation for the binary search algorithm along with the sorting the input elements if they are not sorted.		
3	1. Write a python program to implement bubble sort where the input elements are taken from the user. 2. Write a python program to implement insertion sort where the input elements are taken from the user. 3. Write a python program to implement selection sort where the input elements are taken from the user.	14-17	
4	1. Write a python program to implement merge sort where the input elements are taken from the user 2. Write a python program to implement quick sort where the input elements are taken from the user	18-21	
5	a) Write a Python program to implement a Singly Linked List (SLL) with the following operations: 1. Create LL 2. Insert at Beginning 3. Insert at End 4. Insert at Index 5. Delete by Value 6. Delete Prior Node 7. Delete Last Node 8. Count Number of Nodes 9. Display/Traverse 10. Exit	22-41	

	b) Write a Python program to implement a Double Linked List (DLL) with the following operations: **1. Create LL** **2. Insert at Beginning** **3. Insert at End** **4. Insert at Index** **5. Delete by Value** **6. Delete Prior Node** **7. Delete Last Node** **8. Count Number of Nodes** **9. Display/Traverse** **10. Exit** **c) Write a Python program to implement a Circular Linked List (CLL) with the following operations:** **1. Create LL** **2. Insert at Beginning** **3. Insert at End** **4. Insert at Index** **5. Delete by Value** **6. Delete Prior Node** **7. Delete Last Node** **8. Count Number of Nodes** **9. Display/Traverse** **10. Exit**		
6	**a) write a Python program to implement stack operations such as Push, Pop, Peek, and Display using an Array.** **b)Write a Python program to implement stack operations such as Push, Pop, Peek, and Display using an Linked List**	42-46	
7.	**7a) Write a python program to implement Queue and its operations such as enqueue ,dequeue, peek, display with a menu driven approach. Implement using array and linked list data structures** **7b) Write a python program to implement circular Queue and its operations such as enqueue ,dequeue, peek, display with a menu driven approach. Implement using array and linked list data structures**	47-56	

8.	**8)write a python program to implement a binary tree with its operations using a array and a linked list**	57-62	
9.	**9)write a python program to implement a binary search treee with its operation and traversals**	63-67	
10.	10)**Python program to implement AVL trees.**	68-70	
11.	11)Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing)	71-73	

Week-1

(a) A rocket launch system displays a countdown before launch. Instead of loop implement countdown using recursion. Write a recursive Python program to display a countdown from n to 1, followed by “Launch!”.

CODE:



Output:



Description:

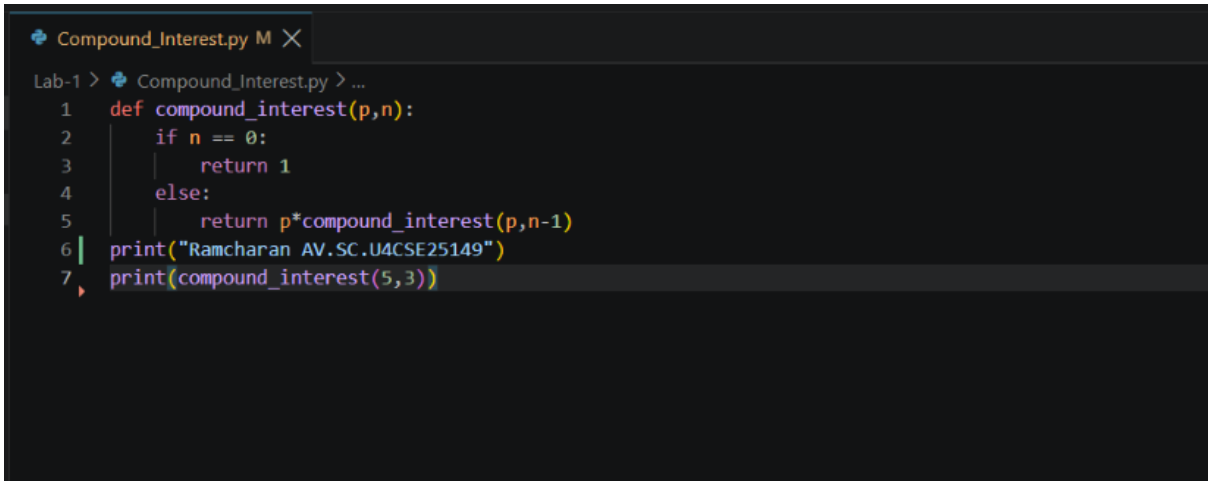
This program displays a rocket launch countdown from n to 1 using recursion instead of loops. The function prints the current countdown number and recursively calls itself with the value decreased by 1. When the value reaches 0, the message “Launch!” is displayed, terminating the recursion.

Error Table:

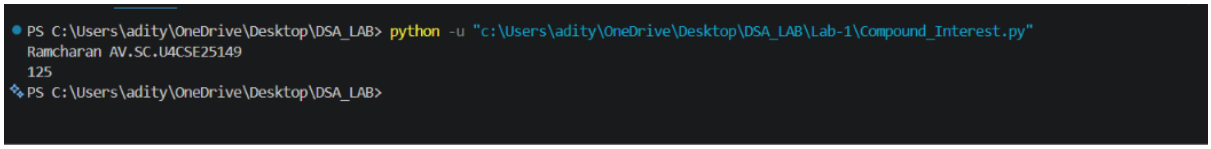
Error	**Rectification**
1. Infinite loop **2. Calling function at wrong place**	**1. Iteration at correct place** **2. Placed with correct syntax**

(b) Banks calculate the growth of an investment using compound interest. The formulae require calculating powers, which can be implemented recursively. Write a recursive program to calculate P^N where:
P=Principal Growth, N=number of years.

CODE:



Output:



Description:

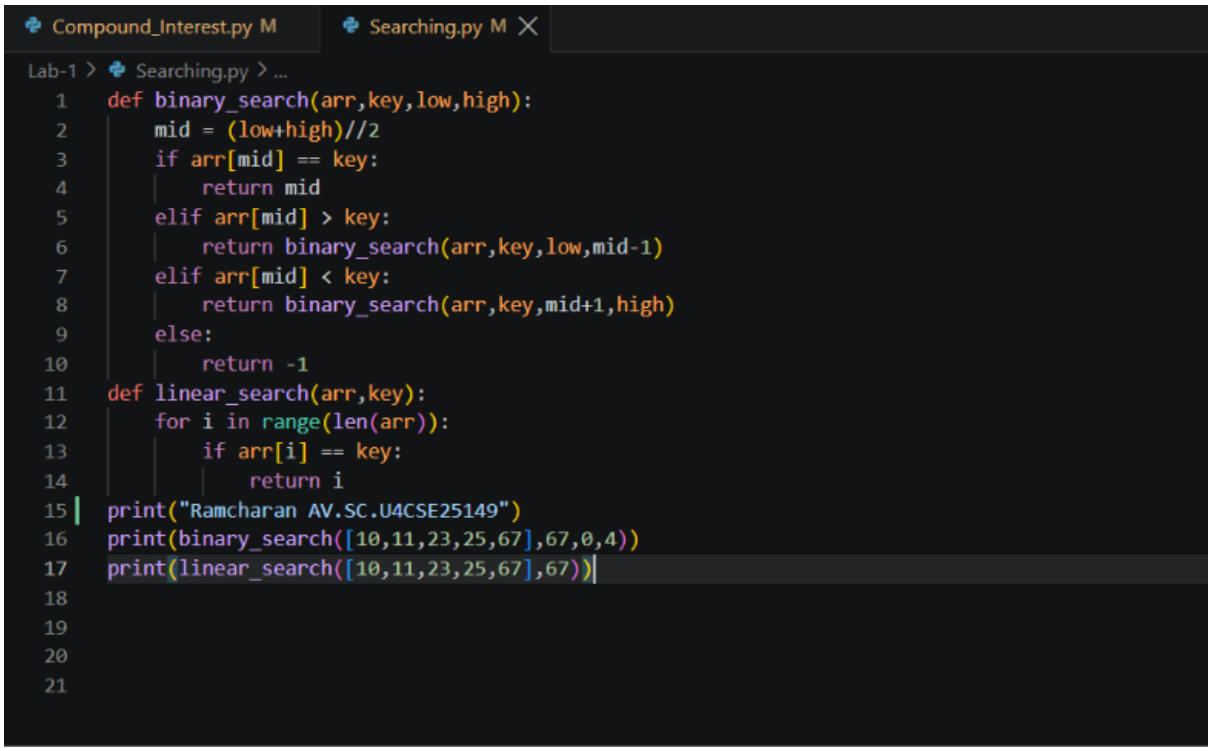
This program calculates P^N using recursion. The function repeatedly multiplies the principal growth factor P by itself while decreasing the value of n by 1. The recursion stops when n becomes 0, since any number raised to power 0 is 1. The final result represents the growth factor of the investment over n years.

Error Table:

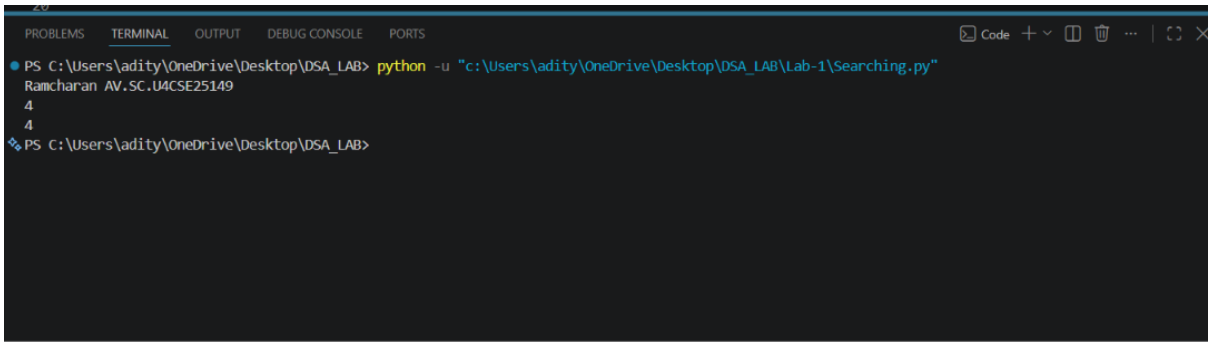
Error	**Rectification**
1. Return 0 **2. print**	**1. Return 1** **2. return**

(c) An HR department stores ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in the list.

CODE:



Output:

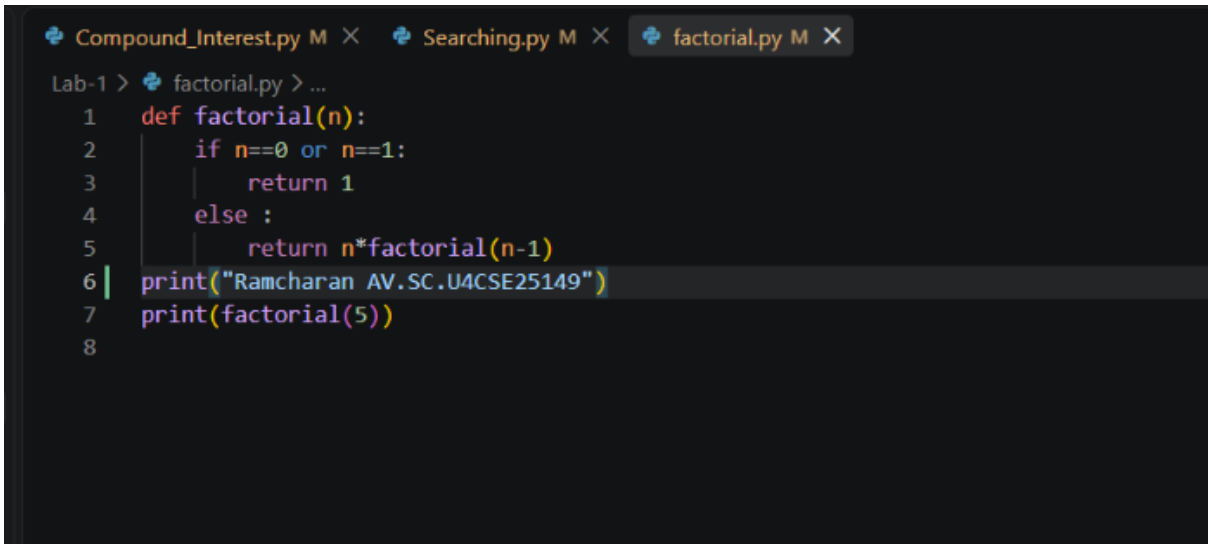


Description:

This program searches for a given employee ID in a list using recursion. The function checks each element of the list one by one and recursively moves to the next index until the employee ID is found, or the end of the list is reached. If the ID is found, its index is returned; otherwise, -1 is returned.

(d) In logistics company, develop a recursive program to calculate the factorial of a given number.

CODE:



Output:



Description:

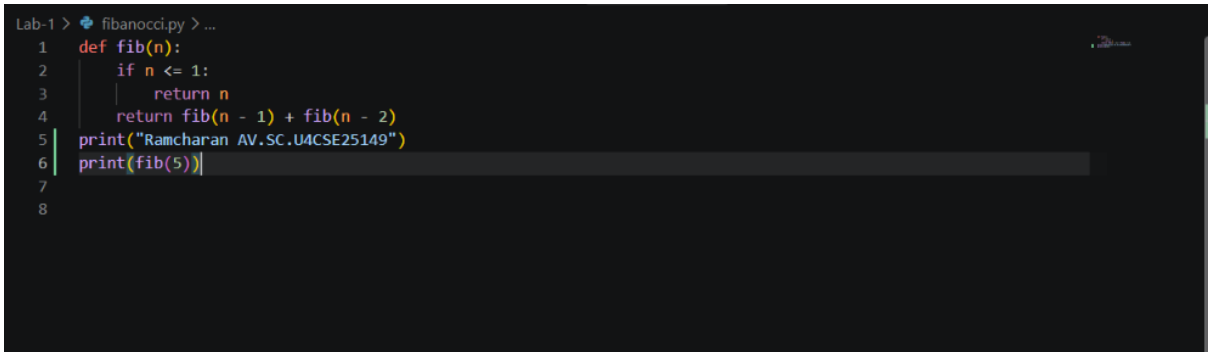
This program calculates the factorial of a given number using recursion. The function multiplies the current number by the factorial of the previous number. The recursion stops when the number becomes 0 or 1, as the factorial of both 0 and 1 is 1.

Error Table:

Error	**Rectification**
1. Emp lst **2. print**	**1. Emp_list** **2. return**

(e) Write a recursive program to print the first n terms of the Fibonacci series

Code:



Output:



Description:

This program generates the first n terms of the Fibonacci series using recursion. Each Fibonacci term is calculated as the sum of the previous two terms. The base cases are F(0)=0 and F(1)=1. The program recursively calculates and displays each term.

Error Table:

Error	**Rectification**
1. n+1 **2. Syntax error**	**1. n-1** **2. rectified**

Week-2

(a). Linear Search

Code:



Output:



Description:

Linear Search is a searching algorithm that checks each element of a list sequentially from the beginning until the required element is found, or the end of the list is reached. It can be used on both sorted and unsorted lists. The time complexity of Linear Search is O(n).

Error Table:

Error	**Rectification**
1. Return placed in wrong place **2. Calling the function**	**1. Corrected later** **2. Rectified**

(b). Binary Search

Code:

```python
def binary_search(arr,key):
    low = 0
    high = len(arr)-1
    while low<=high:
        mid = (low+high)//2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid -1
    return -1
print("Ramcharan AV.SC.U4CSE25149")
n = int(input("Enter the number of elements in the array: "))
print("Enter the Elements: ")
arr = []
for i in range(n):
    arr.append(int(input(" ")))
if arr != sorted(arr):
    print("Sorting the array...")
    print("The array is sorted.")
    arr.sort()
key = int(input("Enter the key you want to search: "))
a = binary_search(arr,key)
print(f"The element {key} is found at {a+1} position in the array")
```

Output:

```
Problems  Terminal  Output  Debug Console  Ports

PS C:\Users\adity\OneDrive\Desktop\DSA_LAB> python -u "c:\Users\adity\OneDrive\Desktop\DSA_LAB\Lab-2\binary_search.py"
Ramcharan AV.SC.U4CSE25149
Enter the number of elements in the array: 10
Enter the Elements:
 2
 12
 32
 1
 4
 21
 14
 18
 4
 5
Sorting the array...
The array is sorted.
Enter the key you want to search: 4
The element 4 is found at 3 position in the array
PS C:\Users\adity\OneDrive\Desktop\DSA_LAB>
```

Description:

Binary Search is an efficient searching algorithm used on a sorted list. It compares the target element with the middle element of the list and eliminates half of the search space after each comparison. This process is repeated until the element is found or the search range becomes empty. The time complexity of Binary Search is O(log n).

Error table:

Error	Rectification
1. Mistakes in main logic **2. Errors in conditions**	**1. Rectified** **2. Rectified**

Week-3

(a). Bubble Sort

Code:

bubble_sort.py

```
Lab-3 > bubble_sort.py
def bubble_sort(arr):
    n = len(arr)
    for i in range(n-1):
        for j in range(n-i-1):
            if arr[j] > arr[j+1]:
                arr[j],arr[j+1] = arr[j+1],arr[j]
    return arr
print("Ramcharan AV.SC.U4CSE25149")
arr=[]
n = int(input("Enter the numnber of elements:"))
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
print("Unsorted array:", arr)
print("Sorted array:", bubble_sort(arr))
```

Output:

```
PS C:\Users\adity\OneDrive\Desktop\DSA_LAB> python -u "c:\Users\adity\OneDrive\Desktop\DSA_LAB\Lab-3\bubble_sort.py"
Ramcharan AV.SC.U4CSE25149
Enter the numnber of elements:5
Enter the elements:
12
3
15
2
10
Unsorted array: [12, 3, 15, 2, 10]
Sorted array: [2, 3, 10, 12, 15]
PS C:\Users\adity\OneDrive\Desktop\DSA_LAB>
```

Description:

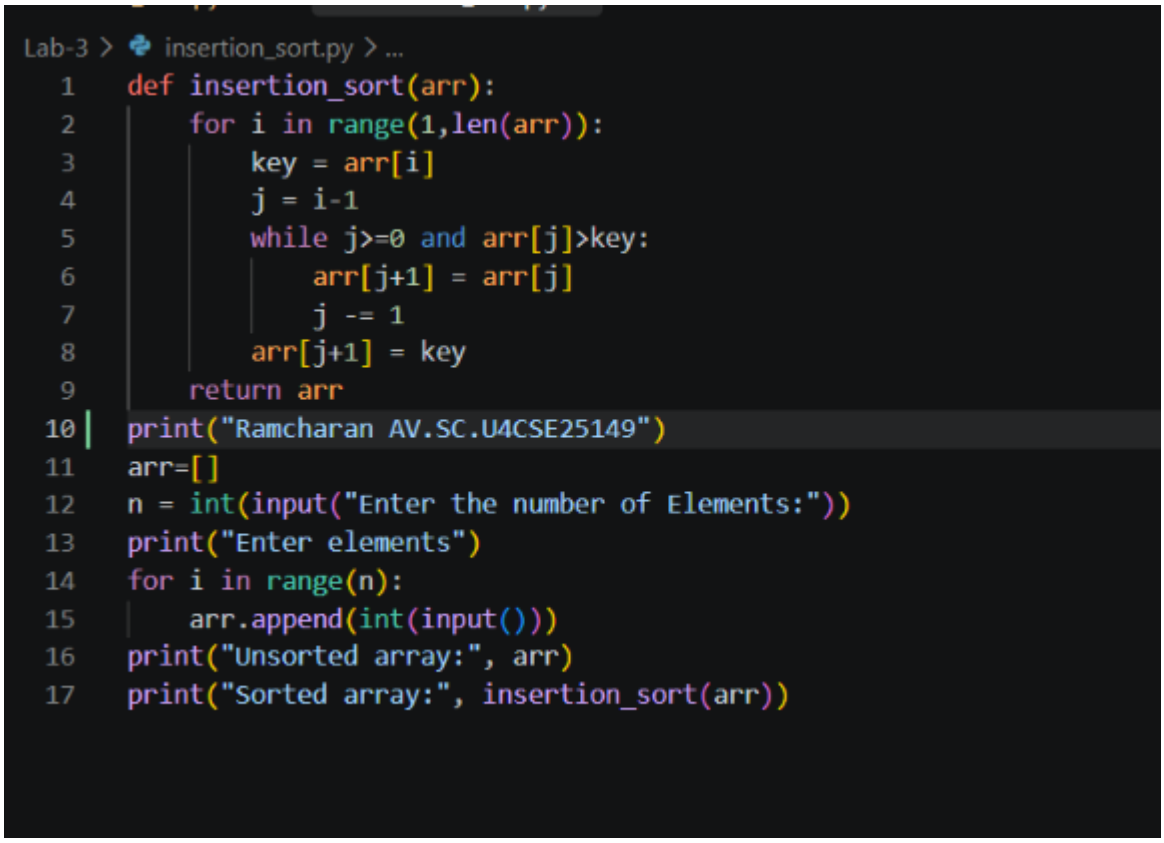
Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. During each pass, the largest unsorted element moves to the end of the list. The process is repeated until the complete list is sorted. The time complexity of Bubble Sort is O (n^2).

Error Table:

Error	Rectification
1. Mistakes in main logic **2. Errors in conditions**	**1. Rectified** **2. Rectified**

(b) Insertion sort

Code:



```
Lab-3 > insertion_sort.py > ...
def insertion_sort(arr):
    for i in range(1,len(arr)):
        key = arr[i]
        j = i-1
        while j>=0 and arr[j]>key:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
    return arr
print("Ramcharan AV.SC.U4CSE25149")
arr=[]
n = int(input("Enter the number of Elements:"))
print("Enter elements")
for i in range(n):
    arr.append(int(input()))
print("Unsorted array:", arr)
print("Sorted array:", insertion_sort(arr))
```

Output:



```
PS C:\Users\adity\OneDrive\Desktop\DSA_LAB> python -u "c:\Users\adity\OneDrive\Desktop\DSA_LAB\Lab-3\insertion_sort.py"
Ramcharan AV.SC.U4CSE25149
Enter the number of Elements:5
Enter elements
12
3
15
9
10
Unsorted array: [12, 3, 15, 9, 10]
Sorted array: [3, 9, 10, 12, 15]
PS C:\Users\adity\OneDrive\Desktop\DSA_LAB>
```

Description:

Insertion Sort is a sorting algorithm that builds the sorted list one element at a time. It takes each element from the unsorted portion and inserts it into its correct position in the sorted portion by shifting larger elements to the right. The process continues until all elements are sorted. The worst-case time complexity of Insertion Sort is O(n^2).

Error table:

Error	Rectification
1. Mistakes in main logic 2. Errors in conditions	1. Rectified 2. Rectified

(c) Selection Sort

Code:

```
Lab-3 > Selection_Sort.py > ...
def Selection_Sort(arr):
    n = len(arr)
    for i in range(n-1):
        min_index = i
        for j in range(i+1,n):
            if arr[j]<arr[min_index]:
                min_index = j
        arr[i],arr[min_index] = arr[min_index],arr[i]
    return arr
print("Ramcharan AV.SC.U4CSE25149")
arr = []
n = int(input("Enter the number of elements:"))
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
print("Unsorted array:", arr)
print("Sorted array:", Selection_Sort(arr))
```

Output:

```
PS C:\Users\adity\OneDrive\Desktop\DSA_LAB> python -u "c:\Users\adity\OneDrive\Desktop\DSA_LAB\Lab-3\Selection_Sort.py"
Ramcharan AV.SC.U4CSE25149
Enter the number of elements:5
Enter the elements:
12
3
15
9
10
Unsorted array: [12, 3, 15, 9, 10]
Sorted array: [3, 9, 10, 12, 15]
PS C:\Users\adity\OneDrive\Desktop\DSA_LAB>
```

Description:

Selection Sort repeatedly finds the smallest element from the unsorted portion of the list and places it at the beginning of that portion. This process is repeated until all elements are arranged in ascending order. The time complexity of Selection Sort is O(n^2).

Error table:

Error	Rectification
1. Mistakes in main logic 2. Errors in conditions	1. Rectified 2. Rectified

Week-4

4a) Write a python program to implement merge sort where the input elements are taken from the user

Program:

```
Lab-4 > merge_sort.py > ...
  1  # Ramcharan AV.SC.U4CSE25149
  2  def merge_sort(elements):
  3      if len(elements) <= 1:
  4          return elements
  5
  6      middle = len(elements) // 2
  7      left = merge_sort(elements[:middle])
  8      right = merge_sort(elements[middle:])
  9
 10      merged = []
 11      left_index = right_index = 0
 12
 13      while left_index < len(left) and right_index < len(right):
 14          if left[left_index] <= right[right_index]:
 15              merged.append(left[left_index])
 16              left_index += 1
 17          else:
 18              merged.append(right[right_index])
 19              right_index += 1
 20
 21      merged.extend(left[left_index:])
 22      merged.extend(right[right_index:])
 23      return merged
 24
 25  print("Ramcharan AV.SC.U4CSE25149")
 26  elements = list(map(int, input("Enter elements separated by spaces: ").split()))
 27  print("Sorted elements:", *merge_sort(elements))
 28
```

Output:

```
PS C:\Users\adity\OneDrive\Desktop\DSA_LAB> python -u "c:\Users\adity\OneDrive\Desktop\DSA_LAB\Lab-4\merge_sort.py"
Ramcharan AV.SC.U4CSE25149
Enter elements separated by spaces: 12 24 2 10 15 21 9 20
Sorted elements: 2 9 10 12 15 20 21 24
PS C:\Users\adity\OneDrive\Desktop\DSA_LAB>
```

Error Table:

Error	Rectification
RecursionError: maximum recursion depth exceeded	base case in merge_sort() len(arr) <1 to len(arr) <= 1
PS C:\Users\Lab\Documents\cse25142\dsa-lab\week-4> python merge_sort.py [2, 3, 7, 7, 8, 3, 7, 7, 8, 7, 7, 8]	kept the extend in the while loop instead of out of while loop

Description:

- The program implements the **Merge Sort** algorithm.
- The user enters the size of the array.
- The elements are taken from the user and stored in a list.
- The merge_sort() function recursively divides the array into two halves.
- The array is divided until each sub-array contains one or zero elements.
- The left and right portions are then sorted recursively.
- The sorted left and right arrays are compared element by element.
- The smaller element is placed into the original array.
- Remaining elements from either portion are then copied into the array.
- Finally, the completely sorted array is displayed.
- Merge Sort follows the **divide-and-conquer** approach.

4b) Write a python program to implement quick sort where the input elements are taken from the user.

Program:

Lab-4 > quick_sort.py > ...

```python
# Ramcharan AV.SC.U4CSE25149
def quick_sort(values):
    """Sort a list in ascending order using quick sort."""
    def partition(low, high):
        pivot = values[high]
        i = low - 1

        for j in range(low, high):
            if values[j] <= pivot:
                i += 1
                values[i], values[j] = values[j], values[i]

        values[i + 1], values[high] = values[high], values[i + 1]
        return i + 1

    def sort(low, high):
        if low < high:
            pivot_index = partition(low, high)
            sort(low, pivot_index - 1)
            sort(pivot_index + 1, high)

    sort(0, len(values) - 1)
    return values

print("Ramcharan AV.SC.U4CSE25149")
elements = input("Enter elements separated by spaces: ").split()
numbers = [int(element) for element in elements]
print("Sorted elements:", *quick_sort(numbers))
```

Output:

```
PS C:\Users\adity\OneDrive\Desktop\DSA_LAB> python -u "c:\Users\adity\OneDrive\Desktop\DSA_LAB\Lab-4\quick_sort.py"
Ramcharan AV.SC.U4CSE25149
Enter elements separated by spaces: 10 2 12 24 8 14 6
Sorted elements: 2 6 8 10 12 14 24
```

Error Table:

Error	Rectification
PS C:\Users\Lab\Documents\cse25142\dsa-lab\week-4> python quick_sort.py Traceback (most recent call last): File "C:\Users\Lab\Documents\cse25142\dsa-lab\week-4\quick_sort.py", line 24, in <module> main() File "C:\Users\Lab\Documents\cse25142\dsa-lab\week-4\quick_sort.py", line 5, in main quick_sort(arr) TypeError: quick_sort() missing 2 required positional arguments: 'low' and 'high' PS C:\Users\Lab\Documents\cse25142\dsa-lab\week-4>	added the two parameters to the function

Description:

The program implements the **Quick Sort** algorithm.
The user enters the size of the array.
The elements are taken from the user and stored in a list.
The quick_sort() function is used to sort the array recursively.
The first element of the current portion is selected as the **pivot**.
Two pointers, i and j, are used to compare elements with the pivot.
i moves from the left side and searches for an element greater than or equal to the pivot.
j moves from the right side and searches for an element smaller than or equal to the pivot.
If i < j, the two elements are swapped.
After the partitioning process, the pivot is placed in its correct position.
The array is then divided into left and right portions.
Quick Sort is recursively applied to both portions.
Finally, the sorted array is printed.

Week-5

5a) Write a Python program to implement a Singly Linked List (SLL) with the following operations:

1. Create LL

2. Insert at Beginning

3. Insert at End

4. Insert at Index

5. Delete by Value

6. Delete Prior Node

7. Delete Last Node

8. Count Number of Nodes

9. Display/Traverse

10. Exit

Program:

```
def main():
    print("CSE25149  T.RAMCHARAN REDDY\n")
    my  LinkedList = None
    while True:
        print(
            "1.create\n2.insert at beginning\n3.insert at end\n4.insert at certain position\n5.delete first
node\n6.delete last node\n7.delete specific node\n"
            "8.count\n9.display\n10.exit"
        )
        option = input("Enter your choice: ")
        if option == "1":
            my_LinkedList = LinkedList()
            print("linked list successfully created")
        elif my_LinkedList is None:
            print("please create the linked list first (option 1)")
        elif option == "2":
            my_LinkedList.insert_begin(int(input("Enter the data: ")))
            print("element is inserted")
        elif option == "3":
            my_LinkedList.insert_end(int(input("Enter the data: ")))
            print("element inserted")
        elif option == "4":
            my_LinkedList.insert_position(
                int(input("Enter the data: ")), int(input("Enter the position: "))
```

```
            )
        elif option == "5":
            my_LinkedList.delete_begin()
        elif option == "6":
            my_LinkedList.delete_end()
        elif option == "7":
            my_LinkedList.delete_position(int(input("Enter position: ")))
        elif option == "8":
            count = my_LinkedList.size()
            print("the size of linked list is {}".format(count))
        elif option == "9":
            my_LinkedList.display()
        elif option == "10":
            print("exiting the program")
            break
        else:
            print("option not defined")


class LinkedList:
    def __init__(self):
        self.head = None

    def insert_begin(self, data):
        node = Node(data)
        if self.head is None:
            self.head = node
            return
        node.next = self.head
        self.head = node

    def insert  end(self, data):
        if self.head is None:
            self.insert  begin(data)
            return
        node = Node(data)
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = node

    def size(self):
        temp = self.head
        count = 0
        while temp:
            temp = temp.next
            count += 1
        return count
```

```
def insert_position(self, data, pos):
    # positions are 1-indexed: pos=1 means new head
    n = self.size()
    if pos < 1 or pos > n + 1:
        print("position not available")
        return
    if pos == 1:
        self.insert_begin(data)
        print("element inserted")
        return
    node = Node(data)
    temp = self.head
    count = 1
    while count < pos - 1:
        temp = temp.next
        count += 1
    node.next = temp.next
    temp.next = node
    print("element inserted")

def display(self):
    if self.head is None:
        print("The linked list is empty")
        return
    temp = self.head
    while temp:
        print(temp.data, end=" <-> ")
        temp = temp.next
    print("None")

def delete  begin(self):
    if self.head is None:
        print("The Linked list is empty")
        return
    self.head = self.head.next
    print("first node deleted")

def delete_end(self):
    if self.head is None:
        print("The Linked list is empty")
        return
    if self.head.next is None:
        self.head = None
        print("last node deleted")
        return
    temp = self.head
    while temp.next.next:
        temp = temp.next
```

```
        temp.next = None
        print("last node deleted")

    def delete_position(self, pos):
        # positions are 1-indexed: pos=1 means head
        if self.head is None:
            print("Empty linked list")
            return
        n = self.size()
        if pos < 1 or pos > n:
            print("position does not exist")
            return
        if pos == 1:
            self.head = self.head.next
            print("node at position is deleted")
            return
        temp = self.head
        count = 1
        while count < pos - 1:
            temp = temp.next
            count += 1
        temp.next = temp.next.next
        print("node at position is deleted")


class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


if __name__ == "__main__":
    main()
```

Output:

```
PS C:\Users\adity\OneDrive\Desktop\DSA_LAB> python -u "c:\Users\adity\OneDrive\Desktop\DSA_LAB\Lab-5\SLL.PY"
CSE25149  T.RAMCHARAN REDDY

1.create
2.insert at beginning
3.insert at end
4.insert at certain position
5.delete first node
6.delete last node
7.delete specific node
8.count
9.display
10.exit
Enter your choice: 1
linked list successfully created
1.create
2.insert at beginning
3.insert at end
4.insert at certain position
5.delete first node
6.delete last node
7.delete specific node
8.count
9.display
10.exit
Enter your choice: 2
Enter the data: 34
element is inserted
1.create
2.insert at beginning
3.insert at end
4.insert at certain position
5.delete first node
6.delete last node
7.delete specific node
8.count
9.display
10.exit
Enter your choice: 3
Enter the data: 23
element inserted
1.create
```

Ln 16, Col 24 Spaces: 4 UTF-8 CRLF { } Python

Problems Terminal Output Debug Console Ports Code

```
PS C:\Users\adity\OneDrive\Desktop\DSA_LAB> python -u "c:\Users\adity\OneDrive\Desktop\DSA_LAB\Lab-5\SLL.PY"
Enter your choice: 3
Enter the data: 23
element inserted
1.create
2.insert at beginning
3.insert at end
4.insert at certain position
5.delete first node
6.delete last node
7.delete specific node
8.count
9.display
10.exit
Enter your choice: 4
Enter the data: 2
Enter the position: 12
position not available
1.create
2.insert at beginning
3.insert at end
4.insert at certain position
5.delete first node
6.delete last node
7.delete specific node
8.count
9.display
10.exit
Enter your choice: 9
34 <-> 23 <-> None
1.create
2.insert at beginning
3.insert at end
4.insert at certain position
5.delete first node
6.delete last node
7.delete specific node
8.count
9.display
10.exit
Enter your choice: 10
exiting the program
```

Ln 16, Col 24 Spaces: 4 UTF-8 CRLF { } Python

Error Table:

Error	Rectification
The program may display an incorrect message for insertion at a specific index even when the index is invalid	**Check whether the index is valid before displaying the successful insertion message.**

Description:

- The program implements a Singly Linked List using Python classes.
- The Node class is used to create individual nodes.
- Each node contains two parts: data and next.
- data stores the value of the node.
- next stores the reference to the next node.
- The SingleLinkedList class manages the linked list.
- The head variable stores the reference to the first node.
- The program provides an option to create a linked list.
- It supports insertion at the beginning of the list.
- It supports insertion at the end of the list.
- It supports insertion at a specific index.
- It supports deletion of a specific value.
- It supports deletion of the first node.
- It supports deletion of the last node.
- It provides a function to count the number of nodes.
- It provides a function to display/traverse all the nodes.
- A menu-driven approach is used to allow the user to select different operations.
- The program continues running until the user selects the Exit option

5b) Write a Python program to implement a Double Linked List (DLL) with the following operations:

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit

Program:

```
def main():
    print("CSE25149 T.RAMCHARAN REDDY\n")
    dll = None
    while True:
        print("\n---menu---\n1.Create DLL\n2.insert begin\n3.insert end"
              "\n4.insert index\n5.delete begin\n6.delete end\n7.delete value\n8.size of DLL\n9.exit")
        o = input("Enter a choice: ")
        match o:
            case '1':
                dll = DLL()
                dll.create()
            case '2':
                if dll is None:
                    print("please create the DLL first")
                else:
                    dll.insert  begin(int(input("Enter the data: ")))
            case '3':
                if dll is None:
                    print("please create the DLL first")
                else:
                    dll.insert_end(int(input("Enter the data: ")))
            case '4':
                if dll is None:
                    print("please create the DLL first")
                else:
                    data = int(input("Enter the data: "))
```

```
                index = int(input("Enter the index: "))
                dll.insert_index(data, index)
            case '5':
                if dll is None:
                    print("please create the DLL first")
                else:
                    dll.delete_begin()
            case '6':
                if dll is None:
                    print("please create the DLL first")
                else:
                    dll.delete_end()
            case '7':
                if dll is None:
                    print("please create the DLL first")
                else:
                    dll.delete_value(int(input("Enter the value: ")))
            case '8':
                if dll is None:
                    print("please create the DLL first")
                else:
                    dll.size()
            case '9':
                break
            case  :
                print("invalid choice")
    print("Exiting the program.....")


class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None


class DLL:
    def    init   (self):
        self.head = None

    def create(self):
        n = int(input("Enter the no of nodes: "))
        for i in range(n):
            data = int(input("Enter the data: "))
            new = Node(data)
            if self.head is None:
                self.head = new
                continue          # fixed: continue, not return
            temp = self.head
```

```
        while temp.next:
            temp = temp.next
        temp.next = new
        new.prev = temp
    return 1

def insert_begin(self, data):
    new = Node(data)
    if self.head is None:
        self.head = new
        return
    self.head.prev = new
    new.next = self.head
    self.head = new

def insert_end(self, data):
    new = Node(data)
    if self.head is None:
        self.insert_begin(data)
        return
    temp = self.head
    while temp.next:
        temp = temp.next
    temp.next = new
    new.prev = temp

def insert  index(self, data, index):
    if index < 0:
        print("not possible")
        return              # fixed: stop execution here
    new = Node(data)
    if index == 0:
        self.insert  begin(data)
        return
    temp = self.head
    for i in range(index - 1):
        if temp is None:
            print("Not available")
            return
        temp = temp.next
    if temp is None:
        print("Not available")
        return
    if temp.next:
        temp.next.prev = new
    new.next = temp.next
    temp.next = new
    new.prev = temp
```

```
def size(self):
    count = 0
    if self.head is None:
        print("List is Empty")
        return count
    temp = self.head
    while temp:
        count += 1
        temp = temp.next
    print(f"the dll contains {count} elements")
    return count

def delete_begin(self):
    if self.head is None:
        print("Empty list")
        return -1
    if self.head.next is None:
        self.head = None
        return 1
    self.head = self.head.next
    self.head.prev = None
    return 1

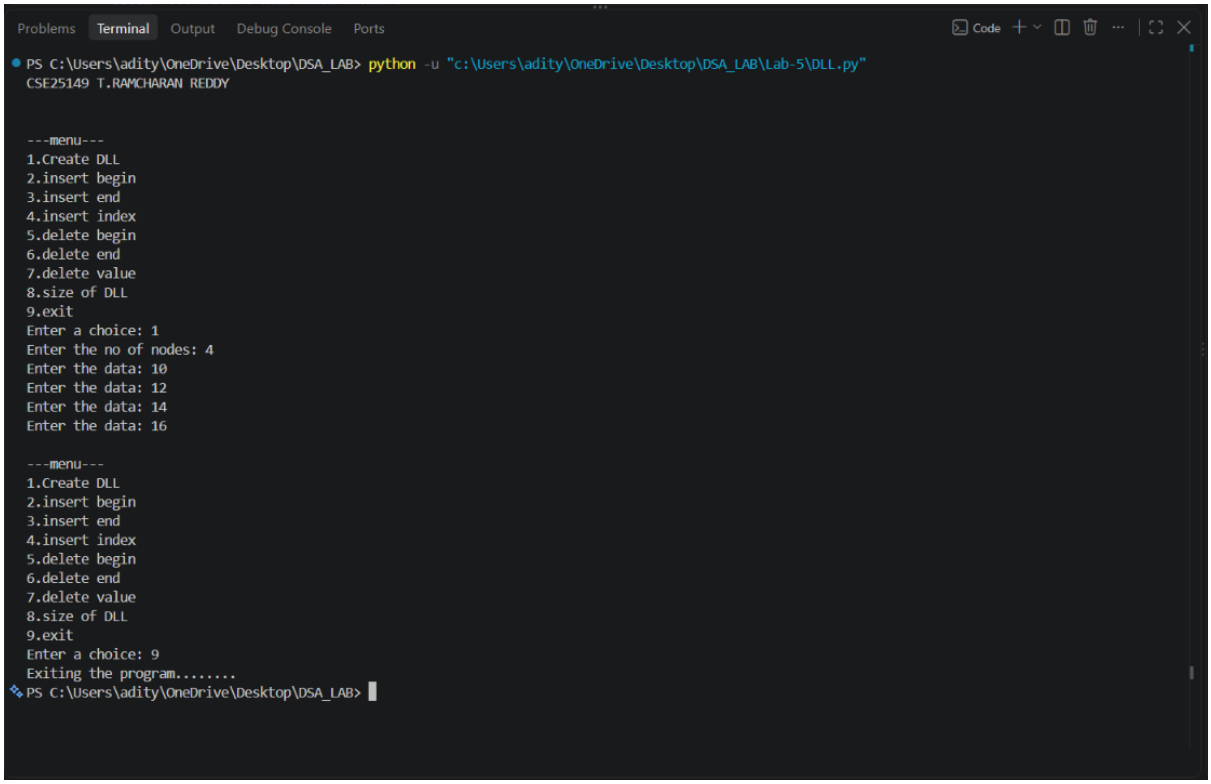
def delete  end(self):
    if self.head is None:
        print("Empty list")          # fixed: no crash on empty list
        return -1
    if self.head.next is None:
        self.head = None
        return 1
    temp = self.head
    while temp.next:
        temp = temp.next
    temp.prev.next = None
    temp.prev = None
    return 1

def delete  value(self, value):
    if self.head is None:
        print("The list is empty")
        return -1
    if self.head.data == value:
        self.head = self.head.next
        if self.head:
            self.head.prev = None      # fixed: clear prev on new head
        return 1
    temp = self.head
```

```
        while temp.next and temp.next.data != value:
            temp = temp.next
        if temp.next is None:
            print("Value not available")
            return -1
        target = temp.next
        temp.next = target.next          # fixed: relink around target, not temp.prev
        if target.next:
            target.next.prev = temp
        return 1


if __name__ == '__main__':
    main()
```

Output:



Error Table:

Error	Rectification
The prev and next links may not be updated correctly during insertion or deletion.	Update both links whenever a node is inserted or deleted.

Description:

- The program implements a Doubly Linked List using Python classes.
- The Node class is used to create individual nodes.
- Each node contains three parts: data, prev, and next.
- data stores the value of the node.
- prev stores the reference to the previous node.
- next stores the reference to the next node.
- The DoubleLinkedList class is used to manage the linked list.
- The head variable stores the reference to the first node.
- The program provides an option to create a doubly linked list.
- It supports insertion at the beginning of the list.
- It supports insertion at the end of the list.
- It supports insertion at a specific index.
- It supports deletion of a specific value.
- It supports deletion of the first node.
- It supports deletion of the last node.
- It provides a function to count the number of nodes.
- It provides a function to display/traverse the list.
- During insertion and deletion, both prev and next links are updated.
- The list is displayed using <--> to show the connection between nodes.
- A menu-driven approach is used to perform different DLL operations.
- The program continues until the user selects the Exit option.

5c) Write a Python program to implement a Circular Linked List (CLL) with the following operations:

1. Create LL

2. Insert at Beginning

3. Insert at End

4. Insert at Index

5. Delete by Value

6. Delete Prior Node

7. Delete Last Node

8. Count Number of Nodes

9. Display/Traverse

10. Exit

Program:

```
def main():
    print("cse25149 T. RAMCHARAN REDDY\n")
    CLL = CircularLinkedList()

    while True:
        print("\n1.Create CLL")
        print("2.Insert at beginning")
        print("3.Insert at ending")
        print("4.Insert at index")
        print("5.Delete by value")
        print("6.Delete first node")
        print("7.Delete last node")
        print("8.Count no of nodes")
        print("9.Display / Traverse")
        print("10.Exit")

        choice = int(input("Enter your choice: "))

        if choice == 1:
            CLL.create()
            print("CLL is created")

        elif choice == 2:
            data = int(input("Enter data to insert: "))
            CLL.insert_begin(data)
            print("Data is inserted at beginning")
```

```
        elif choice == 3:
            data = int(input("Enter data to insert: "))
            CLL.insert_end(data)
            print("Data is inserted at ending")

        elif choice == 4:
            data = int(input("Enter data to insert: "))
            index = int(input("Enter the index: "))
            CLL.insert_index(index, data)

        elif choice == 5:
            data = int(input("Enter data to delete: "))
            CLL.delete(data)

        elif choice == 6:
            CLL.delete_begin()

        elif choice == 7:
            CLL.delete_end()

        elif choice == 8:
            CLL.count()

        elif choice == 9:
            CLL.display()

        elif choice == 10:
            print("Exiting the program")
            break

        else:
            print("Invalid choice")


class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularLinkedList:
    def __init__(self):
        self.head = None

    # Creation
    def create(self):
        n = int(input("Enter no of elements: "))

        for i in range(n):
```

```
        data = int(input("Enter the values: "))
        new = Node(data)

        if self.head is None:
            self.head = new
            new.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            temp.next = new
            new.next = self.head

    # Insert begin
    def insert_begin(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
            new.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            new.next = self.head
            self.head = new
            temp.next = self.head

    # Insert end
    def insert_end(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
            new.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            temp.next = new
            new.next = self.head
```

```
    # Insert at specific index
    def insert_index(self, index, data):
        if index < 0:
            print("Invalid index")
            return

        if index == 0:
            self.insert_begin(data)
            return

        if self.head is None:
            print("Invalid index")
            return

        temp = self.head

        for i in range(index - 1):
            if temp.next == self.head:
                print("Invalid index")
                return

            temp = temp.next

        new = Node(data)
        new.next = temp.next
        temp.next = new

    # Delete specific value
    def delete(self, data):
        if self.head is None:
            print("No data")
            return

        # If head contains the value
        if self.head.data == data:

            # Only one node
            if self.head.next == self.head:
                self.head = None
                print("Value deleted")
                return

            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            temp.next = self.head.next
```

```
            self.head = self.head.next

            print("Value deleted")
            return

        temp = self.head

        while temp.next != self.head and temp.next.data != data:
            temp = temp.next

        if temp.next == self.head:
            print("Value not present")
        else:
            temp.next = temp.next.next
            print("Value deleted")

    # Delete begin
    def delete_begin(self):
        if self.head is None:
            print("No data to delete")
            return

        # Only one node
        if self.head.next == self.head:
            print("Deleted value = ", self.head.data)
            self.head = None
            return

        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        print("Deleted value = ", self.head.data)

        temp.next = self.head.next
        self.head = self.head.next

    # Delete end
    def delete  end(self):
        if self.head is None:
            print("No data to delete")
            return

        # Only one node
        if self.head.next == self.head:
            print("Deleted value = ", self.head.data)
            self.head = None
```

```
            return

        temp = self.head

        while temp.next.next != self.head:
            temp = temp.next

        print("Deleted value = ", temp.next.data)

        temp.next = self.head

    # Count
    def count(self):
        if self.head is None:
            print("No data to count")
            return

        count = 0
        temp = self.head

        while True:
            count += 1
            temp = temp.next

            if temp == self.head:
                break

        print(f"Number of nodes = {count}")

    # Display / Traverse
    def display(self):
        if self.head is None:
            print("No data")
            return

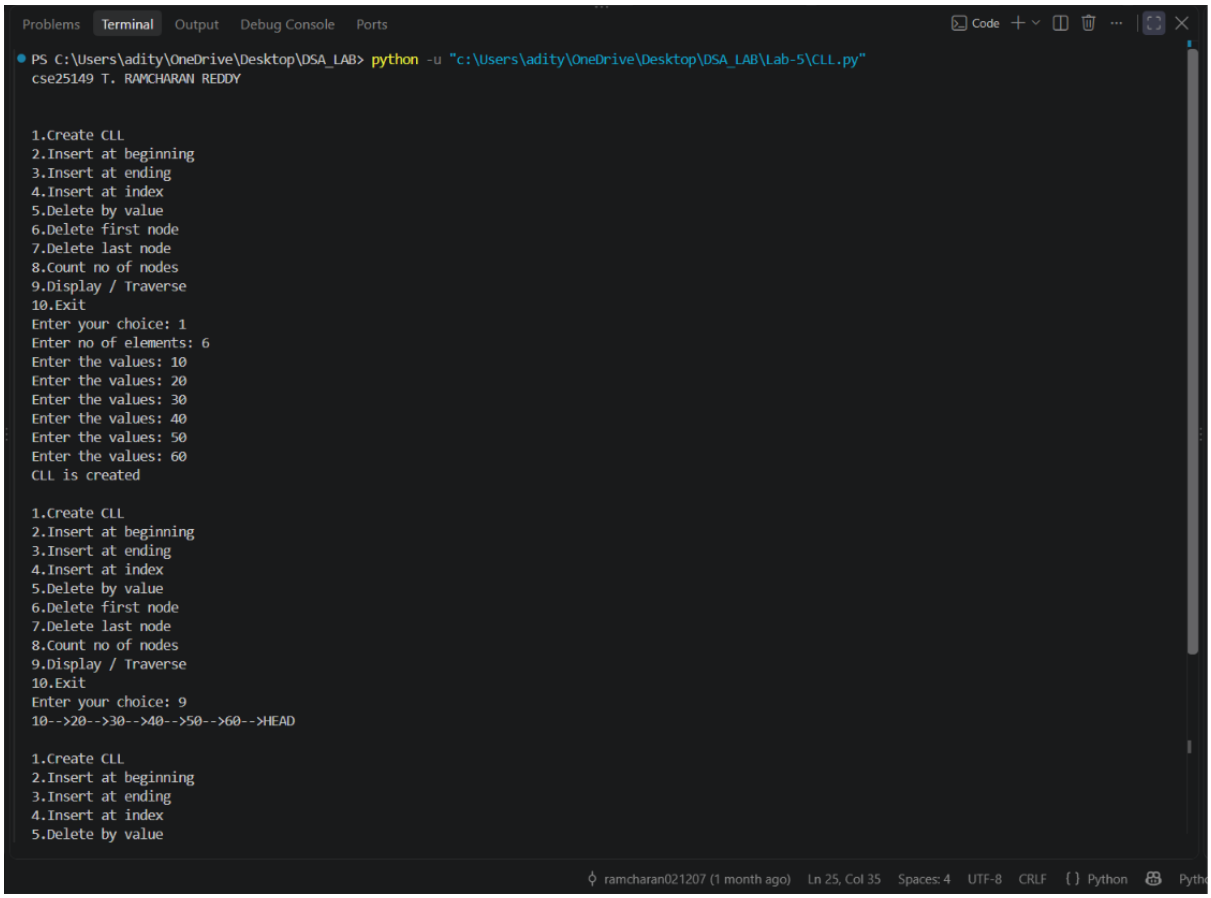
        temp = self.head

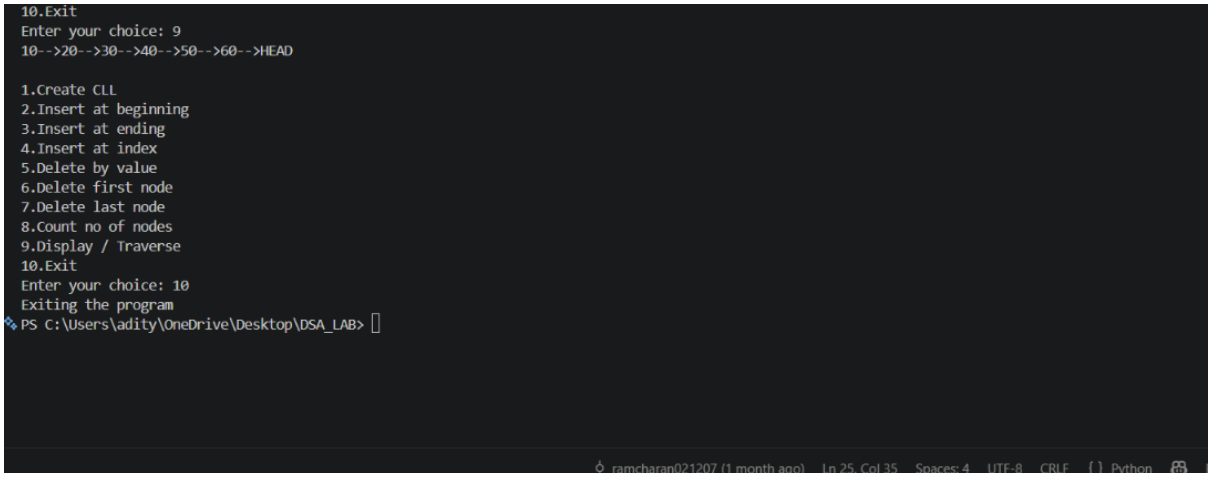
        while True:
            print(temp.data, end="-->")
            temp = temp.next

            if temp == self.head:
                break

        print("HEAD")
if __name__ == '__main__':
    main()
```

Output:


Problems
Terminal
Output
Debug Console
Ports
Code
PS C:\Users\adity\OneDrive\Desktop\DSA_LAB> python -u "c:\Users\adity\OneDrive\Desktop\DSA_LAB\Lab-5\CLL.py"
cse25149 T. RAMCHARAN REDDY
1.Create CLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice: 1
Enter no of elements: 6
Enter the values: 10
Enter the values: 20
Enter the values: 30
Enter the values: 40
Enter the values: 50
Enter the values: 60
CLL is created
1.Create CLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice: 9
10-->20-->30-->40-->50-->60-->HEAD
1.Create CLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
ramcharan021207 (1 month ago)
Ln 25, Col 35
Spaces: 4
UTF-8
CRLF
{} Python


10.Exit
Enter your choice: 9
10-->20-->30-->40-->50-->60-->HEAD
1.Create CLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice: 10
Exiting the program
PS C:\Users\adity\OneDrive\Desktop\DSA_LAB>

Error Table:

Error	Rectification
No errors	

Description:

- The program implements a Circular Linked List using Python classes.
- The Node class is used to create individual nodes.
- Each node contains two parts: data and next.
- data stores the value of the node.
- next stores the reference to the next node.
- In a Circular Linked List, the last node points back to the first node.
- The CircularLinkedList class is used to manage the linked list.
- The head variable stores the reference to the first node.
- The program provides an option to create a circular linked list.
- It supports insertion at the beginning of the list.
- It supports insertion at the end of the list.
- It supports insertion at a specific index.
- It supports deletion of a specific value.
- It supports deletion of the first node.
- It supports deletion of the last node.
- It provides a function to count the number of nodes.
- It provides a function to display/traverse the circular linked list.
- During traversal, the program stops when it reaches the head again.
- The list is displayed using --> and ends with HEAD, showing its circular nature.
- The program handles the special case where the list contains only one node.
- A menu-driven approach is used to perform different CLL operations.
- The program continues until the user selects the Exit option.

Week-6

6a) Write a Python program to implement stack operations such as Push, Pop, Peek, and Display using an Array.

Program:

```
# Ramcharan AV.SC.U4CSE25149
class Stack:
    def __init__(self, capacity):
        self.items = [None] * capacity
        self.capacity = capacity
        self.top = -1

    def push(self, value):
        if self.top == self.capacity - 1:
            print("Stack Overflow")
            return
        self.top += 1
        self.items[self.top] = value
        print(f"{value} pushed onto the stack")

    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
            return
        value = self.items[self.top]
        self.items[self.top] = None
        self.top -= 1
        print(f"Popped element: {value}")

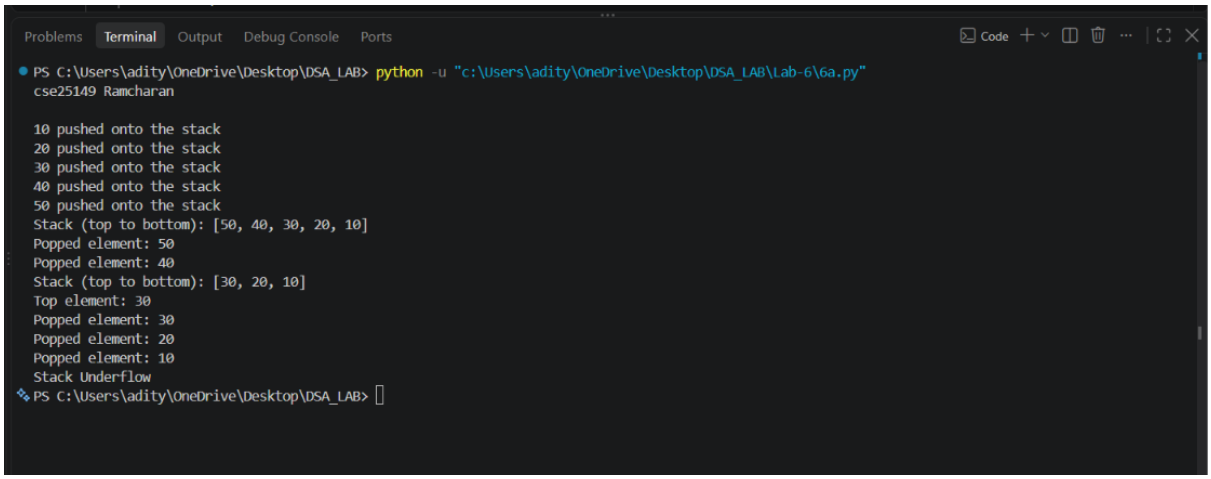
    def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print(f"Top element: {self.items[self.top]}")

    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Stack (top to bottom):", self.items[self.top::-1])

print("cse25149 Ramcharan\n")
stack = Stack(5)
stack.push(10)
stack.push(20)
stack.push(30)
stack.push(40)
stack.push(50)
stack.display()
```

```
stack.pop()
stack.pop()
stack.display()
stack.peek()
stack.pop()
stack.pop()
stack.pop()
stack.pop()
```

Output:



Error Table:

Error	Rectification
In peek, self.peek is taken in ‘if’ does not give correct answer for stack underflow.	Self.top is taken in ‘if’ to give correct output.

Description:

- The Stack class implements a stack using a Python list.
- size stores the maximum size of the stack.
- stack stores the stack elements.
- top keeps track of the top position. Initially, top = -1, meaning the stack is empty.
- **Push** adds an element to the top of the stack.
- **Pop** removes the top element from the stack.
- **Peek** displays the top element without removing it.
- **Display** prints all elements from top to bottom.

- **Stack Overflow** occurs when trying to push into a full stack.
- **Stack Underflow** occurs when trying to pop from an empty stack.
- The program uses a menu-driven approach with options for Push, Pop, Peek, Display, and Exit.

6b) Write a Python program to implement stack operations such as Push, Pop, Peek, and Display using an Linked List

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Stack:
    def __init__(self):
        self.top = None

    def push(self, data):
        node = Node(data)
        node.next = self.top
        self.top = node
        print(f"{data} pushed onto the stack.")

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
            return
        data = self.top.data
        self.top = self.top.next
        print(f"{data} popped from the stack.")

    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
            return
        current = self.top
        print("Stack elements:")
        while current is not None:
            print(current.data, end=" ")
```

```
            current = current.next
        print()

print("cse25149 Ramcharan\n")
stack = Stack()

stack.push(10)
stack.push(20)
stack.push(30)
stack.push(40)
stack.push(50)
stack.display()
stack.pop()
stack.pop()
stack.display()
stack.peek()
stack.pop()
stack.pop()
stack.pop()
stack.pop()
```

Error-Table:

Error	Rectification
No errors	

Outut:



Description:

- The stack is implemented using a **singly linked list**.

- Each node contains data and a next pointer.
- top points to the top node of the stack.
- **Push** inserts a new node at the beginning of the linked list.
- **Pop** removes the node pointed to by top.
- **Peek** displays the data of the top node without removing it.
- **Display** traverses the linked list from top to the last node.
- If top is None, the stack is empty.
- Stack Overflow is not a fixed-size issue here because the stack grows dynamically, subject to available memory.

Week-7

7a)

Program:

```
# queue using array.
# cse25149 Ramcharan
class Queue:
    def __init__(self, q_size):
        self.arr = [0] * q_size
        self.size = 0
        self.capacity = q_size
        self.front = 0
    def enqueue(self, x):

        if self.size == self.capacity:
            return

        self.arr[self.size] = x

        self.size += 1
    def dequeue(self):
        if self.size == 0:
            return

        for i in range(1, self.size):
            self.arr[i-1] = self.arr[i]

        self.size -= 1
    def getFront(self):
        if self.size == 0:
            return -1

        return self.arr[self.front]
    def display(self):

        for i in range(self.front, self.size):
            print(self.arr[i], end=" ")
        print()

if __name__ == "__main__":
    q = Queue(4)
    print("cse25149 Ramcharan\n")
    q.enqueue(1)
    q.enqueue(2)
    q.enqueue(3)
    print(q.getFront())
    q.dequeue()
```

```
q.enqueue(4)
q.display()
```

Output:



Description:

This Python program implements a **Queue using an Array (List)**. A queue follows the **FIFO (First In, First Out)** principle, meaning the element inserted first is removed first.

- __init__() creates the queue with a fixed capacity and initializes its size.
- enqueue(x) adds an element to the rear of the queue if the queue is not full.
- dequeue() removes the front element by shifting all remaining elements one position to the left.
- getFront() returns the front element without removing it. It returns -1 if the queue is empty.
- display() prints all the elements currently present in the queue.
- In the main program, a queue of capacity 4 is created, elements 1, 2, 3 are inserted, 1 is removed, and 4 is added.
- The final queue displayed is **2 3 4**.

ErrorTable:

Error	Rectification
No errors	

Output:



Using Linked-List:

Program:

```
# cse25149 Ramcharan
class Node:
    def __init__(self, key):
        self.key = key
        self.next = None

class MyQueue:
    def __init__(self):
        self.front = None
        self.rear = None
        self.size = 0

    def enqueue(self, x):
        temp = Node(x)

        if self.rear is None:
            self.front = temp
            self.rear = temp
        else:
            self.rear.next = temp
            self.rear = temp

        self.size += 1

    def dequeue(self):
        if self.front is None:
            return None
        res = self.front.key

        self.front = self.front.next

        if self.front is None:
            self.rear = None

        self.size -= 1
        return res


    def getFront(self):
        if self.front is None:
            return None
        return self.front.key

    def getRear(self):
        if self.rear is None:
            return None
        return self.rear.key
```

```
    def isEmpty(self):
        return self.front is None

    def getSize(self):
        return self.size


# Example
if __name__ == "__main__":
    queue = MyQueue()
    print("cse25149 Ramcharan\n")
    queue.enqueue(10)
    queue.enqueue(20)
    queue.enqueue(30)

    print(queue.dequeue())

    print(queue.getFront())
    print(queue.getRear())

    print(queue.isEmpty())

    print(queue.getSize())

    print(queue.dequeue())
    print(queue.dequeue())

    print(queue.isEmpty())
```

Output:



Description:

This Python program implements a **Queue using a Linked List**. The queue follows the **FIFO (First In, First Out)** principle, where the element inserted first is removed first.

- Node represents each element of the queue. It stores the **key** and a reference to the **next node**.
- MyQueue maintains two pointers:
  - front → points to the first element.
  - rear → points to the last element.
- enqueue(x) adds a new node at the **rear** of the queue.
- dequeue() removes the node from the **front** and returns its value.
- getFront() returns the first element without removing it.
- getRear() returns the last element without removing it.
- isEmpty() checks whether the queue is empty.
- getSize() returns the number of elements currently in the queue.
- The size variable keeps track of the number of elements in the queue.

In the example, 10, 20, and 30 are inserted. Then 10 is removed, followed by checking the front, rear, size, and whether the queue is empty.

Error Table:

Error	Rectification
No errors	

7b)

Program:

```
# cse25149 Ramcharan
class myQueue:
    def __init__(self, cap):
        self.arr = [0]*cap
        self.front = 0
        self.size = 0
        self.capacity = cap

    def enqueue(self, x):
        if self.size == self.capacity:
            print("Queue is full!")
            return
        rear = (self.front + self.size) % self.capacity
        self.arr[rear] = x
        self.size += 1

    def dequeue(self):
        if self.size == 0:
            print("Queue is empty!")
            return -1
        res = self.arr[self.front]
        self.front = (self.front + 1) % self.capacity
        self.size -= 1
        return res

    def getFront(self):
        if self.size == 0:
            return -1
        return self.arr[self.front]

    def getRear(self):
        if self.size == 0:
            return -1
        rear = (self.front + self.size - 1) % self.capacity
        return self.arr[rear]

if __name__ == "__main__":
    q = myQueue(5)
    print("cse25149 Ramcharan\n")
    q.enqueue(10)
    q.enqueue(20)
    q.enqueue(30)
    print(q.getFront(), q.getRear())
    q.dequeue()
    print(q.getFront(), q.getRear())
```

```
q.enqueue(40)
print(q.getFront(), q.getRear())
```

Output:



Description:

This Python program implements a **Circular Queue using an Array**. A circular queue follows the **FIFO (First In, First Out)** principle and allows the unused spaces at the beginning of the array to be reused.

- __init__() creates an array with a fixed capacity and initializes front and size.
- enqueue(x) adds an element at the rear of the queue. The **modulo % operator** is used to make the rear position circular.
- dequeue() removes the element from the front and moves the front position to the next location using circular movement.
- getFront() returns the first element of the queue without removing it.
- getRear() returns the last element of the queue without removing it.
- If the queue is full, enqueue() displays **"Queue is full!"**.
- If the queue is empty, dequeue() displays **"Queue is empty!"** and returns -1.

Error Table:

Error	Rectification
No errors	

Using Linked List:

Program:

```
# cse25149 Ramcharan
class Node:
    def __init__(self, new_data):
        self.data = new_data
        self.next = None

class Queue:
    def __init__(self):
        self.front = self.rear = None


def enQueue(q, value):
    newNode = Node(value)

    if q.front is None:
        q.front = newNode
    else:
        q.rear.next = newNode

    q.rear = newNode
    q.rear.next = q.front


def deQueue(q):
    if q.front is None:
        return -1

    value = None

    if q.front == q.rear:
        value = q.front.data
        q.front = q.rear = None
    else:
        temp = q.front
        value = temp.data
        q.front = q.front.next
        q.rear.next = q.front

    return value


def front(q):
    front = q.front
```

```
    if front is None:
        return -1

    return front.data


def rear(q):
    rear = q.rear

    if rear is None:
        return -1

    return rear.data

def printQueue(q):
    if q.front is None:
        return

    curr = q.front
    while True:
        print(curr.data, end=" ")
        curr = curr.next
        if curr == q.front:
            break
    print()

if __name__ == "__main__":
    q = Queue()
    print("cse25149 Ramcharan\n")
    enQueue(q, 14)
    enQueue(q, 22)
    enQueue(q, 6)
    enQueue(q, 20)

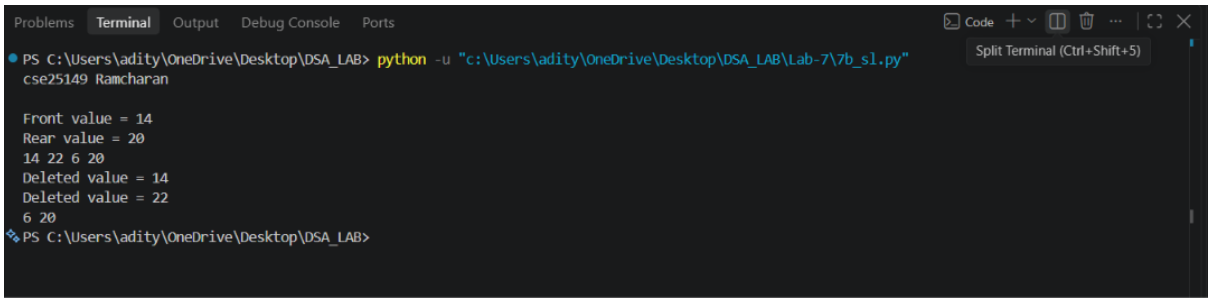
    print("Front value =", front(q))
    print("Rear value =", rear(q))

    printQueue(q)

    print("Deleted value =", deQueue(q))
    print("Deleted value =", deQueue(q))

    printQueue(q)
```

Output:



Description:

This Python program implements a **Circular Queue using a Linked List**. The queue follows the **FIFO (First In, First Out)** principle. Since it is circular, the last node points back to the first node.

- Node represents each element and contains data and a next pointer.
- Queue maintains two pointers:
    - front → points to the first node.
    - rear → points to the last node.
- enQueue(q, value) adds a new node at the rear. The rear node's next pointer is made to point to the front, forming the circular structure.
- deQueue(q) removes the node from the front and updates the circular connection.
- front(q) returns the value of the first element without removing it.
- rear(q) returns the value of the last element without removing it.
- printQueue(q) traverses the circular linked list and prints all elements. It uses a while True loop because there is no None at the end of the list.
- If the queue is empty, the functions return -1.

Error Table:

Error Table:		
S.No	**Error / Issue**	**Correction**
1	**No check for queue size being 0**	**Check that size > 0 before creating the queue.**
2	**int(input()) can cause an error if the user enters a non-numeric value**	**Use valid integer input or handle ValueError.**

Week-8

8(a) Aim: write a python program to implement a binary tree with its operations using a array

Program:

```
#T. RAMCHARAN REDDY
#AV.SC.U4CSE25149
class BinaryTree:
    def __init__(self,size):
        self.tree = [None]*size
        self.size = size
    def insert_root(self,data):
        self.tree[0] = data
    def insert_left(self,parent_index,data):
        child_index = 2*parent_index + 1
        if child_index < self.size:
            self.tree[child_index] = data
        else:
            print("Index out of range.")
    def insert_right(self,parent_index,data):
        child_index = 2*parent_index + 2
        if child_index < self.size:
            self.tree[child_index] = data
        else:
            print("index out of range")
    def pre_trav(self,index=0):
        if index>self.size or self.tree[index] == None:
            return
        print(self.tree[index],end=" ")
        self.pre_trav(2*index+1)
        self.pre_trav(2*index+2)
    def In_trav(self,index=0):
        if index>self.size or self.tree[index] == None:
            return None
        self.In_trav(2*index+1)
        print(self.tree[index],end=" ")
        self.In_trav(2*index+2)
    def post_trav(self,index=0):
        if index>self.size or self.tree[index] == None:
            return
        self.pre_trav(2*index+1)
        self.pre_trav(2*index+2)
        print(self.tree[index],end = " ")
    def level_trav(self):
        for value in self.tree:
            if value is not None:
                print(value,end=" ")
print("T. RAMCHARAN REDDY AV.SC.U4CSE25149")
bt = BinaryTree(20)
bt.insert_root(10)
```

```
bt.insert_left(0,5)
bt.insert_right(1,8)
bt.insert_right(0,16)
bt.insert_left(2,15)
bt.insert_right(2,20)
bt.insert_right(6,14)
print("Preorder")
bt.pre_trav()
print("Inorder")
bt.In_trav()
print("Postorder")
bt.post_trav()
```

Output:



Description:

This Python program implements a Binary Tree using an array representation. It allows the user to insert a root node, left child, and right child using array indices. It also implements four tree traversal techniques: Preorder, Inorder, Postorder, and Level Order Traversal. The left and right child positions are calculated using the formulas 2*i + 1 and 2*i + 2, respectively, where i is the parent node's index. The program uses recursive functions for depth-first traversals and a loop for level order traversal.

Error Table:

Error	Rectification
Using index > self.size as the boundary condition in traversal functions.	Use index >= self.size to prevent accessing an invalid array index.
Accessing self.tree[index] without first checking whether the index is within the valid range.	Check the index boundary before accessing the array element.

8(b) write a python program to implement a binary tree with its operations using a linked list.

Program:

```
print("CSE25149 T.Ramcharan")

class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BinaryTree:
    def __init__(self):
        self.root = None

    # Insert a node
    def insert(self, value):
        new_node = Node(value)

        if self.root is None:
            self.root = new_node
            return

        queue = [self.root]

        while queue:
            current = queue.pop(0)

            if current.left is None:
                current.left = new_node
                return
            else:
                queue.append(current.left)

            if current.right is None:
                current.right = new_node
                return
            else:
                queue.append(current.right)

    # Search a node
    def search(self, value):
        if self.root is None:
            return False

        queue = [self.root]

        while queue:
            current = queue.pop(0)

            if current.data == value:
```

```
            return True

        if current.left:
            queue.append(current.left)

        if current.right:
            queue.append(current.right)

    return False

# Inorder Traversal
def inorder(self, node):
    if node:
        self.inorder(node.left)
        print(node.data, end=" ")
        self.inorder(node.right)

# Preorder Traversal
def preorder(self, node):
    if node:
        print(node.data, end=" ")
        self.preorder(node.left)
        self.preorder(node.right)

# Postorder Traversal
def postorder(self, node):
    if node:
        self.postorder(node.left)
        self.postorder(node.right)
        print(node.data, end=" ")

# Level Order Traversal
def level_order(self):
    if self.root is None:
        return

    queue = [self.root]

    while queue:
        current = queue.pop(0)
        print(current.data, end=" ")

        if current.left:
            queue.append(current.left)

        if current.right:
            queue.append(current.right)

# Main program
bt = BinaryTree()

n = int(input("Enter number of nodes: "))

for i in range(n):
```

```
    value = int(input(f"Enter value for node {i + 1}: "))
    bt.insert(value)

print("\nInorder Traversal:")
bt.inorder(bt.root)

print("\nPreorder Traversal:")
bt.preorder(bt.root)

print("\nPostorder Traversal:")
bt.postorder(bt.root)

print("\nLevel Order Traversal:")
bt.level_order()

key = int(input("\n\nEnter value to search: "))

if bt.search(key):
    print(key, "found in the binary tree")
else:
    print(key, "not found in the binary tree")
```

Output:

```
PS C:\Users\adity\OneDrive\Desktop\DSA_LAB> python -u "c:\Users\adity\OneDrive\Desktop\DSA_LAB\Lab-8\8b.py"
CSE25149 T.Ramcharan
Enter number of nodes: 4
Enter value for node 1: 10
Enter value for node 2: 20
Enter value for node 3: 30
Enter value for node 4: 40

Inorder Traversal:
40 20 10 30
Preorder Traversal:
10 20 40 30
Postorder Traversal:
40 20 30 10
Level Order Traversal:
10 20 30 40

Enter value to search: 20
20 found in the binary tree
PS C:\Users\adity\OneDrive\Desktop\DSA_LAB>
```

Description:

This Python program implements a Binary Tree using a Linked List (Node-based representation). Each node contains data and pointers to its left and right child nodes. The program supports insertion of nodes using Level Order Insertion, searching for a specific element, and performing four types of tree traversals: Inorder, Preorder, Postorder, and Level Order Traversal. A queue is used for insertion, searching, and level order traversal, while recursive functions are used for the depth-first traversal techniques. Finally, the program accepts user input to construct the binary tree and search for a given value.

Error Table:

Error	Rectification
Not initializing the root node before inserting elements.	Initialize self.root = None in the constructor.
Creating a new node without assigning it to the tree.	Assign the new node to the root or an appropriate left/right child.

Week-9

9) **write a python program to implement a binary search treee with its operation and traversals.**

Program:

```
print("CSE25149 T. Ramcharan")
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None

    # Insert a node
    def insert(self, root, value):
        if root is None:
            return Node(value)

        if value < root.data:
            root.left = self.insert(root.left, value)
        elif value > root.data:
            root.right = self.insert(root.right, value)

        return root

    # Search a node
    def search(self, root, key):
        if root is None or root.data == key:
            return root

        if key < root.data:
            return self.search(root.left, key)

        return self.search(root.right, key)

    # Inorder Traversal
    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)

    # Preorder Traversal
    def preorder(self, root):
        if root:
            print(root.data, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)
```

```
    # Postorder Traversal
    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.data, end=" ")

    # Find minimum value
    def minimum(self, root):
        while root.left:
            root = root.left
        return root.data

    # Delete a node
    def delete(self, root, key):
        if root is None:
            return root

        if key < root.data:
            root.left = self.delete(root.left, key)
        elif key > root.data:
            root.right = self.delete(root.right, key)
        else:
            if root.left is None:
                return root.right
            elif root.right is None:
                return root.left

            root.data = self.minimum(root.right)
            root.right = self.delete(root.right, root.data)

        return root

# Main program
tree = BST()

while True:
    print("\n\n1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value to insert: "))
        tree.root = tree.insert(tree.root, value)
        print("Node inserted successfully")

    elif choice == 2:
```

```
        key = int(input("Enter value to search: "))
        if tree.search(tree.root, key):
            print("Element found")
        else:
            print("Element not found")

    elif choice == 3:
        key = int(input("Enter value to delete: "))
        if tree.search(tree.root, key):
            tree.root = tree.delete(tree.root, key)
            print("Node deleted successfully")
        else:
            print("Element not found")

    elif choice == 4:
        print("Inorder Traversal:")
        tree.inorder(tree.root)

    elif choice == 5:
        print("Preorder Traversal:")
        tree.preorder(tree.root)

    elif choice == 6:
        print("Postorder Traversal:")
        tree.postorder(tree.root)

    elif choice == 7:
        print("Program terminated")
        break

    else:
        print("Invalid choice")
```

Output:

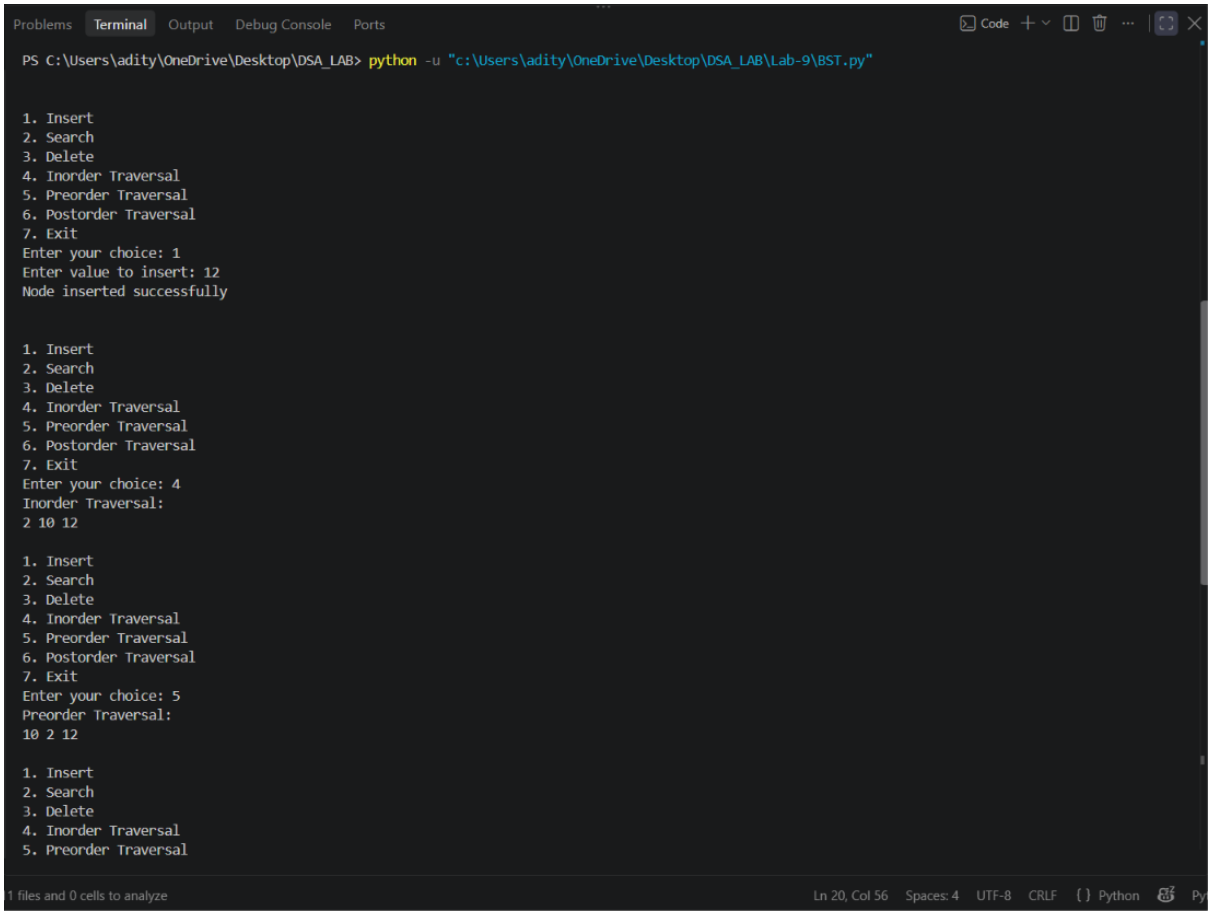
```
Problems  Terminal  Output  Debug Console  Ports                                                    Code
PS C:\Users\adity\OneDrive\Desktop\DSA_LAB> python -u "c:\Users\adity\OneDrive\Desktop\DSA_LAB\Lab-9\BST.py"
CSE25149 T. Ramcharan

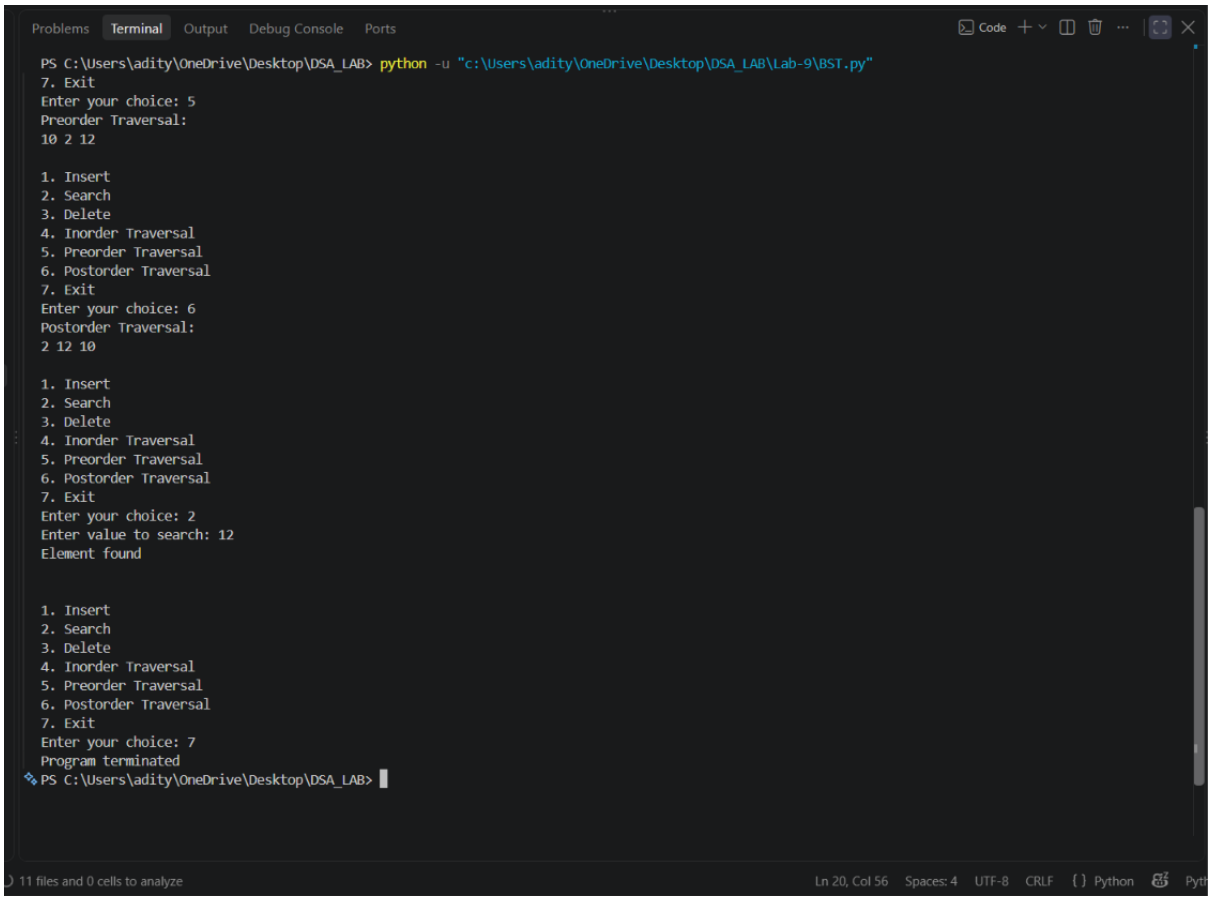

1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 1
Enter value to insert: 10
Node inserted successfully


1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 1
Enter value to insert: 2
Node inserted successfully


1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 1
Enter value to insert: 12
Node inserted successfully


1. Insert
2. Search
11 files and 0 cells to analyze                    Ln 20, Col 56  Spaces: 4  UTF-8  CRLF  { } Python
```


Problems Terminal Output Debug Console Ports
Code
PS C:\Users\adity\OneDrive\Desktop\DSA_LAB> python -u "c:\Users\adity\OneDrive\Desktop\DSA_LAB\Lab-9\BST.py"
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 1
Enter value to insert: 12
Node inserted successfully
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 4
Inorder Traversal:
2 10 12
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 5
Preorder Traversal:
10 2 12
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
1 files and 0 cells to analyze
Ln 20, Col 56 Spaces: 4 UTF-8 CRLF { } Python


Problems Terminal Output Debug Console Ports
Code
PS C:\Users\adity\OneDrive\Desktop\DSA_LAB> python -u "c:\Users\adity\OneDrive\Desktop\DSA_LAB\Lab-9\BST.py"
7. Exit
Enter your choice: 5
Preorder Traversal:
10 2 12
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 6
Postorder Traversal:
2 12 10
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 2
Enter value to search: 12
Element found
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 7
Program terminated
PS C:\Users\adity\OneDrive\Desktop\DSA_LAB>
11 files and 0 cells to analyze
Ln 20, Col 56 Spaces: 4 UTF-8 CRLF { } Python

Description:

This Python program implements a Binary Search Tree (BST) using Linked List representation. Each node contains data and references to its left and right child nodes. The program supports insertion, searching, deletion, and three types of tree traversals: Inorder, Preorder, and Postorder. Recursive functions are used to maintain the BST property, where smaller values are inserted into the left subtree and larger values into the right subtree. The deletion operation handles all three cases: deleting a leaf node, deleting a node with one child, and deleting a node with two children. For a node with two children, the minimum value from the right subtree (inorder successor) replaces the deleted node. A menu-driven interface allows users to perform these operations interactively.

Error Table:

Error	Rectification
Not initializing the root of the BST.	Initialize self.root = None in the constructor.
Inserting smaller and larger elements into incorrect subtrees.	Insert smaller values into the left subtree and larger values into the right subtree. Insert smaller values into the left subtree and larger values into the right subtree.

Week-10

10) **Python program to implement AVL trees.**

Program:

```
print("CSE25149 T. Ramcharan")
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1


class AVLTree:
    def height(self, root):
        if root is None:
            return 0
        return root.height

    def getBalance(self, root):
        if root is None:
            return 0
        return self.height(root.left) - self.height(root.right)

    def rightRotate(self, z):
        y = z.left
        T3 = y.right

        y.right = z
        z.left = T3

        z.height = 1 + max(self.height(z.left),
                           self.height(z.right))
        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))

        return y

    def leftRotate(self, z):
        y = z.right
        T2 = y.left

        y.left = z
        z.right = T2

        z.height = 1 + max(self.height(z.left),
                           self.height(z.right))
```

```
        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))

        return y

    def insert(self, root, key):
        if root is None:
            return Node(key)

        if key < root.key:
            root.left = self.insert(root.left, key)
        elif key > root.key:
            root.right = self.insert(root.right, key)
        else:
            return root

        root.height = 1 + max(self.height(root.left),
                              self.height(root.right))

        balance = self.getBalance(root)

        # LL Case
        if balance > 1 and key < root.left.key:
            return self.rightRotate(root)

        # RR Case
        if balance < -1 and key > root.right.key:
            return self.leftRotate(root)

        # LR Case
        if balance > 1 and key > root.left.key:
            root.left = self.leftRotate(root.left)
            return self.rightRotate(root)

        # RL Case
        if balance < -1 and key < root.right.key:
            root.right = self.rightRotate(root.right)
            return self.leftRotate(root)

        return root

    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.key, end=" ")
            self.inorder(root.right)


# Main Program
```

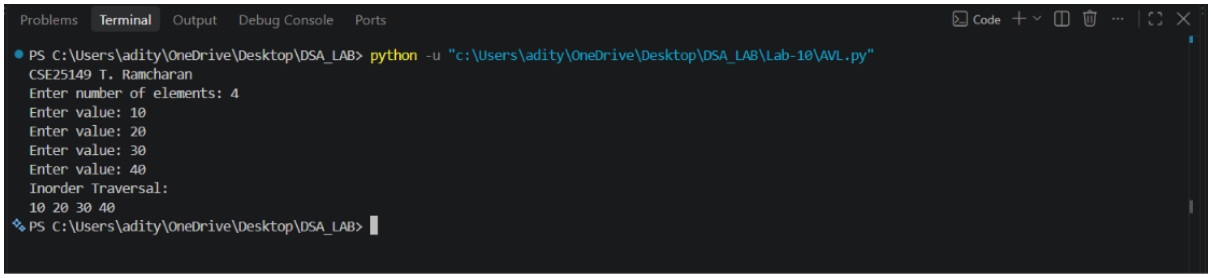
```
tree = AVLTree()
root = None

n = int(input("Enter number of elements: "))

for i in range(n):
    value = int(input("Enter value: "))
    root = tree.insert(root, value)

print("Inorder Traversal:")
tree.inorder(root)
```

Output:



Description:

This Python program implements an AVL Tree (Self-Balancing Binary Search Tree) using Linked List representation. Each node stores a key, references to its left and right children, and its height. The program supports insertion of elements while automatically maintaining the balance of the tree using height calculations and balance factors. The balance factor is calculated as the difference between the heights of the left and right subtrees. If the tree becomes unbalanced after insertion, appropriate rotations (LL, RR, LR, and RL) are performed to restore balance. Finally, the program displays the elements using Inorder Traversal, which prints the keys in ascending order.

Error Table:

Error	Rectification
Not initializing the height of a newly created node.	Initialize self.height = 1 because a new leaf node has height 1.
Incorrect height calculation for a node.	Calculate height using 1 + max(height(left), height(right)).

Week-11

11) **Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).**

Program:

```
print("CSE25149 T. Ramcharan")
def separate_chaining(keys, size):
    table = [[] for i in range(size)]

    for key in keys:
        index = key % size
        table[index].append(key)

    print("\nSeparate Chaining:")
    for i in range(size):
        print(i, ":", table[i])

# Linear Probing
def linear_probing(keys, size):
    table = [None] * size

    for key in keys:
        index = key % size

        while table[index] is not None:
            index = (index + 1) % size

        table[index] = key

    print("\nLinear Probing:")
    for i in range(size):
        print(i, ":", table[i])

# Quadratic Probing
def quadratic_probing(keys, size):
    table = [None] * size

    for key in keys:
        index = key % size
        j = 0

        while j < size:
            new_index = (index + j * j) % size

            if table[new_index] is None:
                table[new_index] = key
                break

            j += 1

        else:
```

```
            print("Cannot insert", key, "- Table is full")

    print("\nQuadratic Probing:")
    for i in range(size):
        print(i, ":", table[i])

# Main Program
size = int(input("Enter hash table size: "))

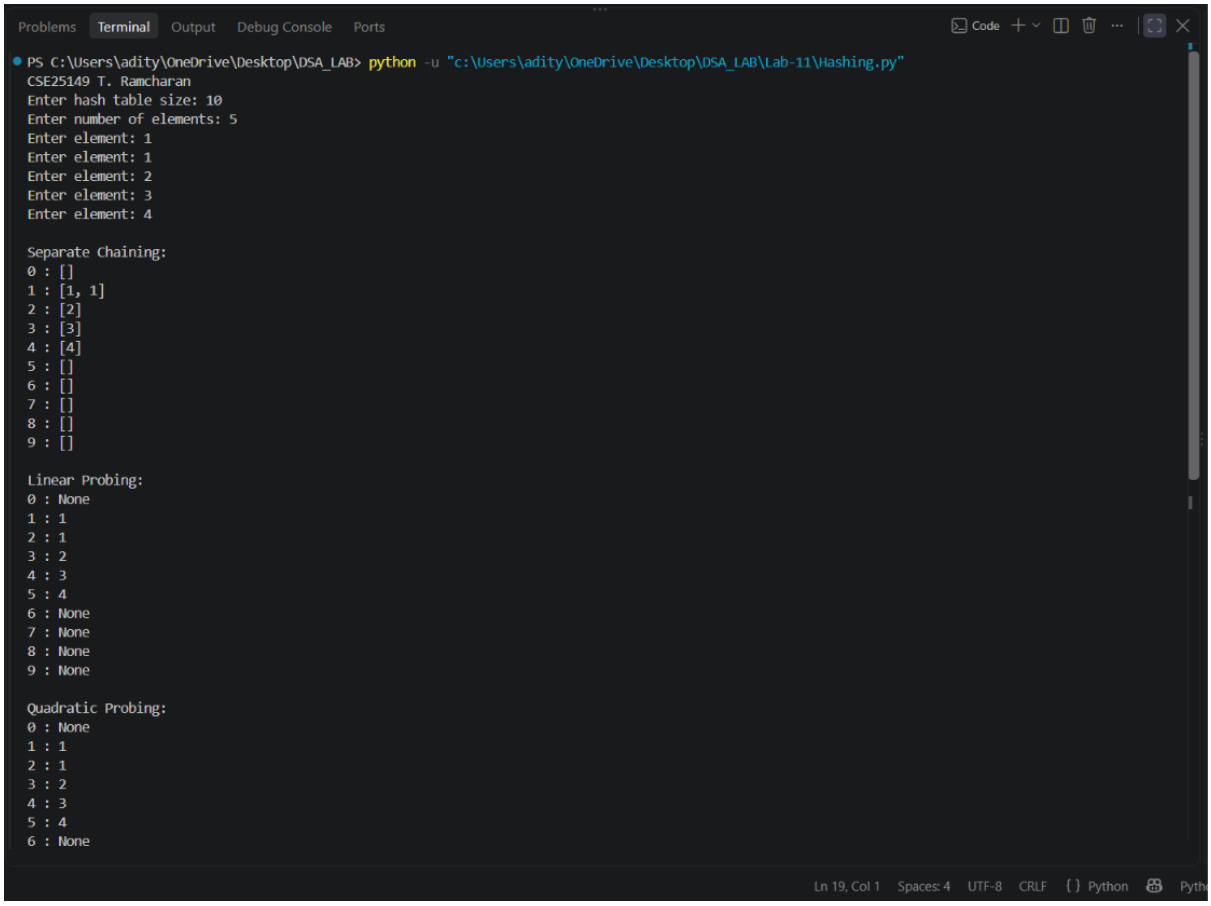
n = int(input("Enter number of elements: "))

keys = []

for i in range(n):
    key = int(input("Enter element: "))
    keys.append(key)

separate_chaining(keys, size)
linear_probing(keys, size)
quadratic_probing(keys, size)
```

Output:



Description:

This Python program implements Hashing Techniques using Separate Chaining, Linear Probing, and Quadratic Probing to handle collisions in a hash table. The hash function key % size is used to calculate the index for each element. In Separate Chaining, collisions are handled by storing multiple elements in a list at the same index. In Linear Probing, the program searches for the next available position sequentially. In Quadratic Probing, it searches for an empty position using the quadratic formula (index + j^2) % size. Finally, the program accepts the hash table size and elements from the user and displays the resulting hash tables for all three collision-resolution techniques.

Error Table:

Error	Rectification
Not initializing the hash table before inserting elements.	Initialize the table using [None] * size for open addressing and [[] for i in range(size)] for separate chaining.
Using an incorrect hash function.	Use key % size to calculate the hash index.